CS228: Introduction to Deep Learning

Lecture 7: PyTorch

Instructor: Zhouxing Shi

PyTorch

- A Python package for deep learning
- Key features:
  - Tensor computation with hardware acceleration (GPU)
  - Autograd: automatically handles the backward pass given the forward pass
  - A large number of deep learning components implemented

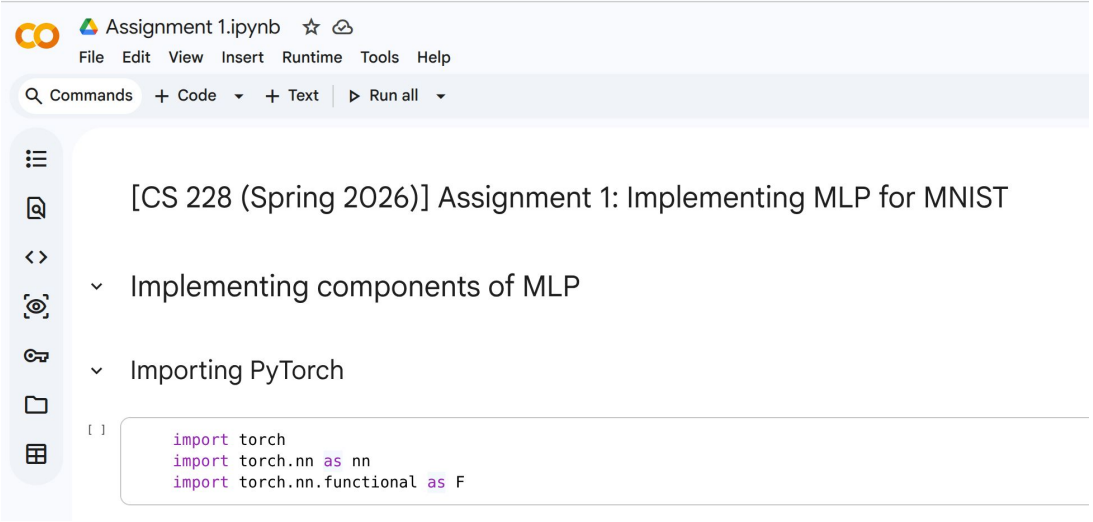
Installing PyTorch

- Local installation: https://pytorch.org/get-started/locally

PyTorch Build	Stable (2.11.0)			Preview (Nightly)	
Your OS	Linux		Mac		Windows
Package	Pip		LibTorch		Source
Language	Python			C++ / Java	
Compute Platform	CUDA 12.6	CUDA 12.8	CUDA 13.0	ROCm 7.2	CPU
Run this Command:	`pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu126`				

Installing PyTorch

- Use PyTorch directly on Google Colab (https://colab.research.google.com/)
  - Online Jupyter Notebook
  - Common libraries pre-installed
  - Free GPUs
  - Sharing



Tensor

- Basic data structure in PyTorch
  - Data with any number of dimensions
  - Generalizes scalars, vectors/arrays, matrices
- Creation: tensor, ones, zeros, randn
- Attributes: shape, dtype, device
- Access and modify values
- Change shapes: reshape, unsqueeze, squeeze, transpose
- Change dtype by “to”
- Change device by “to” or “cpu” / “cuda”

Tensor computation

- Arithmetic operations: +, -, *, /
- Matrix multiplication: matmul
- Comparison operations: <=, >=, <, >
- Logical operations: &, |, ~
- Reduction operations: sum, max, min, mean

Autograd

- A computational graph is automatically built (recorded on a “tape”)
- Get gradients automatically:
  - torch.autograd.grad
  - .backward

Neural Network

- Each neural network model or module is implemented by inheriting “torch.nn.Module”
- Implement “__init__” and “forward” functions
- Leverage existing components:
  - nn.Linear
  - nn.ReLU, nn.Sigmoid
  - nn.CrossEntropyLoss

Optimizer

- Basic optimizer: Stochastic Gradient Descent (SGD)
  - Stochastic: use a random subset (“batch”) of the dataset
  - Better at jumping out of stationary points
- Use torch.optim.SGD

Training pipeline

- Create the model
- Load the dataset and create data loaders with a *batch size*
- Epochs: each *epoch* goes over the training dataset once
- For each iteration in an epoch:
  - Get a batch of data
  - Forward pass to get outputs
  - Calculate the loss function on the outputs
  - Backward pass to get gradients
  - Call the optimizer to update parameters

Resources

- PyTorch tutorials: https://pytorch.org/tutorials/
- PyTorch documentation: https://pytorch.org/docs/stable/index.html